Computational complexity analysis for SpatialZoomer

Because SpatialZoomer consists of multiple computational steps, its runtime and memory usage cannot be fully described by the complexity terms. The following computational complexity analysis is therefore provided as a reference. Overall, the time complexity of SpatialZoomer is approximately $O(nlogn + nm)$ and the space complexity is approximately $O(nm)$, where n is the number of cells and m is the number of genes. Specifically, the computational complexity can be decomposed into five main steps:

Table 1 Time and Space Complexity of Major Steps in SpatialZoomer

Step	Time complexity	Space complexity
Spatial KNN graph construction	$O(nklogn)$, which reduces to $O(nlogn)$ when k is fixed.	$O(nk)$, which reduces to $O(n)$.
NMF on transcriptomics	$O(T_{NMF}nmp)$, which reduces to $O(nm)$ when T_{NMF} has a fixed upper bound and p is fixed.	$O(nm + np + mp)$, dominated by $O(nm)$.
Multi-scale signal processing with heat kernels (Chebyshev approximation)	$O(pqK_{cheby}E)$; since $O(E) = O(nk)$, this reduces to $O(n)$ when k, p, q, K_{cheby} are fixed.	$O(npq)$, which reduces to $O(n)$.
Critical-scale identification by dynamic programming	Correlation matrix calculation: $O(npq^2)$, which reduces to $O(n)$ when p, q are fixed; DP itself is constant with respect to n.	Correlation matrix: $O(pq^2)$; Matrices in DP are constant with respect to n.
Two-step clustering	Standardization: $O(np)$; MiniBatchKMeans: $O(T_{kmeans}bn_{meta}p)$; Leiden/UMAP (bounded by KNN based on metacells): $O(n_{meta}k_{meta}logn_{meta})$ Overall, this reduces to $O(n + n_{meta}logn_{meta})$.	Standardization: $O(np)$; MiniBatchKMeans: $O(n_{meta}p + bp)$; KNN graph construction on metacells: $O(n_{meta}k_{meta})$; Leiden/UMAP: $O(E_{meta} + n_{meta})$. Overall, this reduces to $O(n + n_{meta})$

Here, we define the variables and fixed parameters used in the complexity analysis.

- Variables:

n: number of cells; m: number of genes;

E: number of edges in the spatial neighbor graph, with $O(E) = O(nk) = O(k)$;

n_{meta}: number of K-means clusters (metacells), with $n_{meta} < n$. By default, n_{meta}=10,000;

for smaller datasets (n<10,000), it is set to $n/10$. Therefore, $O(n_{meta}) = O(n)$ in the worst case.

E_{meta}: number of edges in the metacell graph, $O(E) = O(n_{meta}k_{meta}) = O(n_{meta})$.

- Fixed parameters:

k: number of neighbors in spatial KNN graph, $k \ll n$;

k_{meta}: number of neighbors in the metacell KNN graph used for Leiden and UMAP, with $k_{meta} \ll n_{meta}$.

p: number of cell programs / NMF rank, with $p \ll n$.

q: number of candidate scales, with $q \ll n$.

b: MiniBatchKMeans batch size, default 10000.

K_{cheby}: order of Chebyshev polynomial approximation to the heat kernel, default 30.

T_{NMF}, T_{kmeans} iteration numbers, with $T_{NMF} \leq 200$ and $T_{kmeans} \leq 300$.

Under these settings, the overall time complexity is $O(nlogn + nm + n + n_{meta}logn_{meta}) = O(nlogn + nm)$, and the overall space complexity is $O(n + nm + n_{meta}) = O(nm)$.

Consistent with the above analysis, the runtime and memory scaling results on two simulated datasets show that the overall empirical runtime and memory increase approximately linearly with cell number over the tested range (Figure 1). This trend is expected because the $O(nlogn)$ term in the time complexity arises from spatial KNN graph construction, which remains relatively efficient in practice.

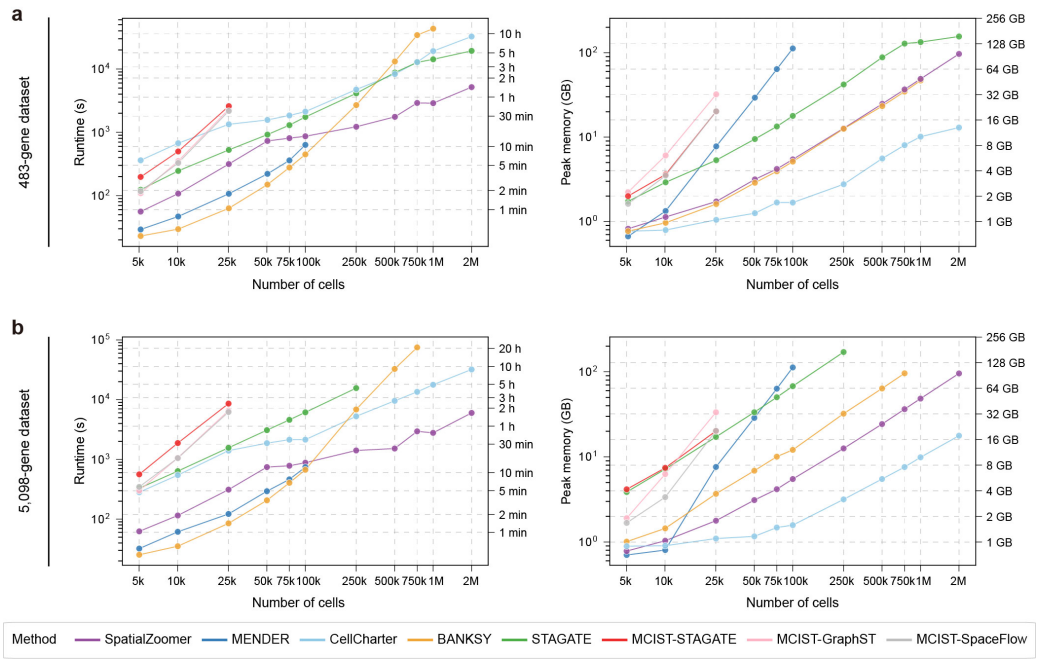

Figure. 1 Benchmark of runtime and peak memory for six methods. a-b, Runtime and peak memory usage of the six methods with increasing cell numbers. MCIST includes three variants

combined with STAGATE, GraphST, and SpaceFlow, respectively. **a**, Dataset constructed from Vizgen MERFISH mouse brain data (483 genes). **b**, Dataset constructed from Xenium data (5,098 genes). Cell numbers were set to 5k, 10k, 25k, 50k, 75k, 100k, 250k, 500k, 750k, 1M, and 2M. All methods were tested on a CPU server with 256 GB of RAM (Server 1 in Supplementary Table 3), using a single clustering resolution for each dataset. Missing results indicate runs that either terminated automatically due to memory overflow or were manually stopped after exceeding 26 hours.